Branch and bound

Branch and bound is a systematic method for solving optimization problems. Branch and bound is a rather general optimization technique that applies where the greedy method and dynamic programming fail.However,it is much lower. Indeed, it often leads to exponential time complexities in the worst case.On the other hand,if applied carefully,it can lead to algorithms that run reasonably fast on average.

Branch-and-bound procedure requires two tools.

1. Branching
   - a way of covering the feasible region by several smaller feasible sub-regions (ideally,splitting into sub-regions).
   - since the procedure may be repeated recursively to each of the sub regions and all produced sub-regions naturally form a tree structure,called search tree or branch-and-bound-tree. Its nodes are the constructed sub-regions.
2. Bounding
   - which is a fast way of finding upper and lower bounds for the optimal solution within a feasible sub-region

The Branch and bound technique like Backtracking explores the implicit graph and deals with the optimal solution to a given problem.In this technique at each stage we calculate the bound for a particular node and check whether this bound will be able to give the solution or not .If we find that at any node the solution so obtained is appropriate but the remaining solution is not leading to a best case then we leave this node without exploring.

Terminologies

- Each node in this tree defines a **problem state**
- All paths from the root to other nodes define the **state space** of the problem
- **Solution states** are those problem states "S" for which the path from the root to 's' defines a tuple in the solution space.
- **Answer states** are those solution states 's' for which the path from the root to "S"defines a tuple that is a member of the set of solutions.
- The tree organization of the solution space referred to as the **state space tree.**
- A node which has been generated and all of whose children have not yet been generated is called a **live node**.
- The live node whose children are currently being generated is called the **E-node** (node being expanded).
- A **dead node** is a generated node which is not to be expanded further or all of whose children have been generated.

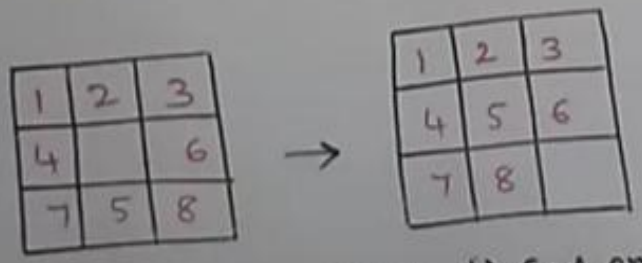
The term branch and bound refers to all state space search methods in which all children of E-node are generated before any other live node can become the E-node.

8-puzzle problem

→ In 8-puzzle problem, there are 8-tiles which are numbered from 1 to 8 placed on a 9-tile capacity square frame.

→ The objective of 8-puzzle problem is to transform the arrangement of tiles from Initial arrangement to a goal arrangement

→ The Initial and Goal arrangement is shown in below figures.



(a) Initial arrangement (b) Goal arrangement

→ There is always an empty slot in the Initial arrangement.

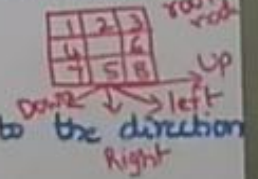
→ Legal moves are the moves in which the tiles adjacent to empty slot are moved to either left, Right, up, Down.

→ The Initial arrangement is called as Initial Stage and goal arrangement is called as goal State.

→ The State space Tree for 8-puzzle is large because, there can be 8! Different arrangements. A partial State space is shown in below fig.

→ In the Statespace Tree, the nodes are numbered as per the level.

→ Each next move is generated based on empty slot positions.

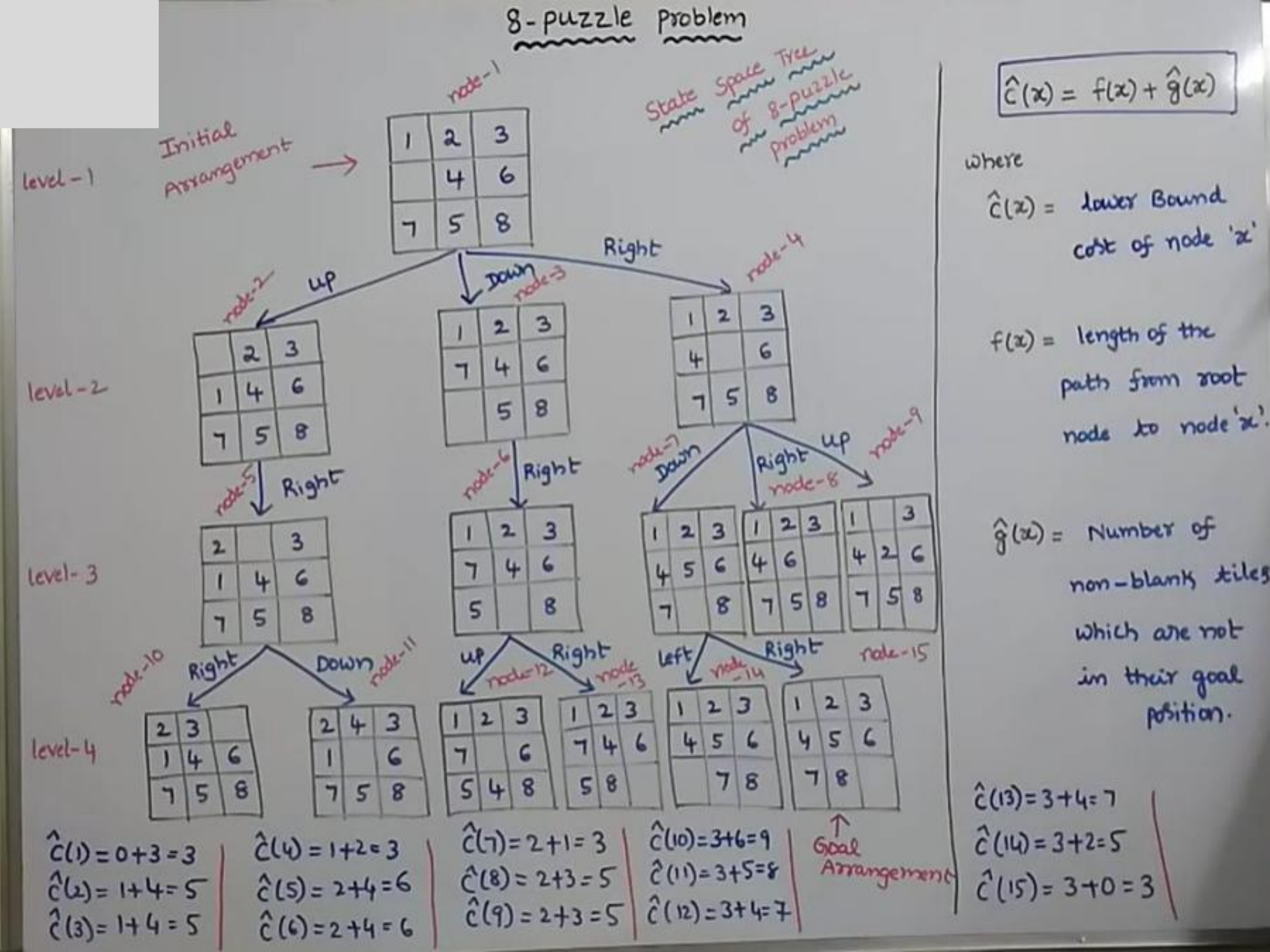


→ Edges are labelled according to the direction in which empty space moves.

→ The root node becomes the E-node.

→ We can decide which node to become an E-node based on estimation formula

8-puzzle Problem



$$\hat{c}(x) = f(x) + \hat{g}(x)$$

where

$\hat{c}(x)$ = lower Bound cost of node 'x'

$f(x)$ = length of the path from root node to node 'x'.

$\hat{g}(x)$ = Number of non-blank tiles which are not in their goal position.

8-puzzle problem using least cost Branch and Bound (LCBB)

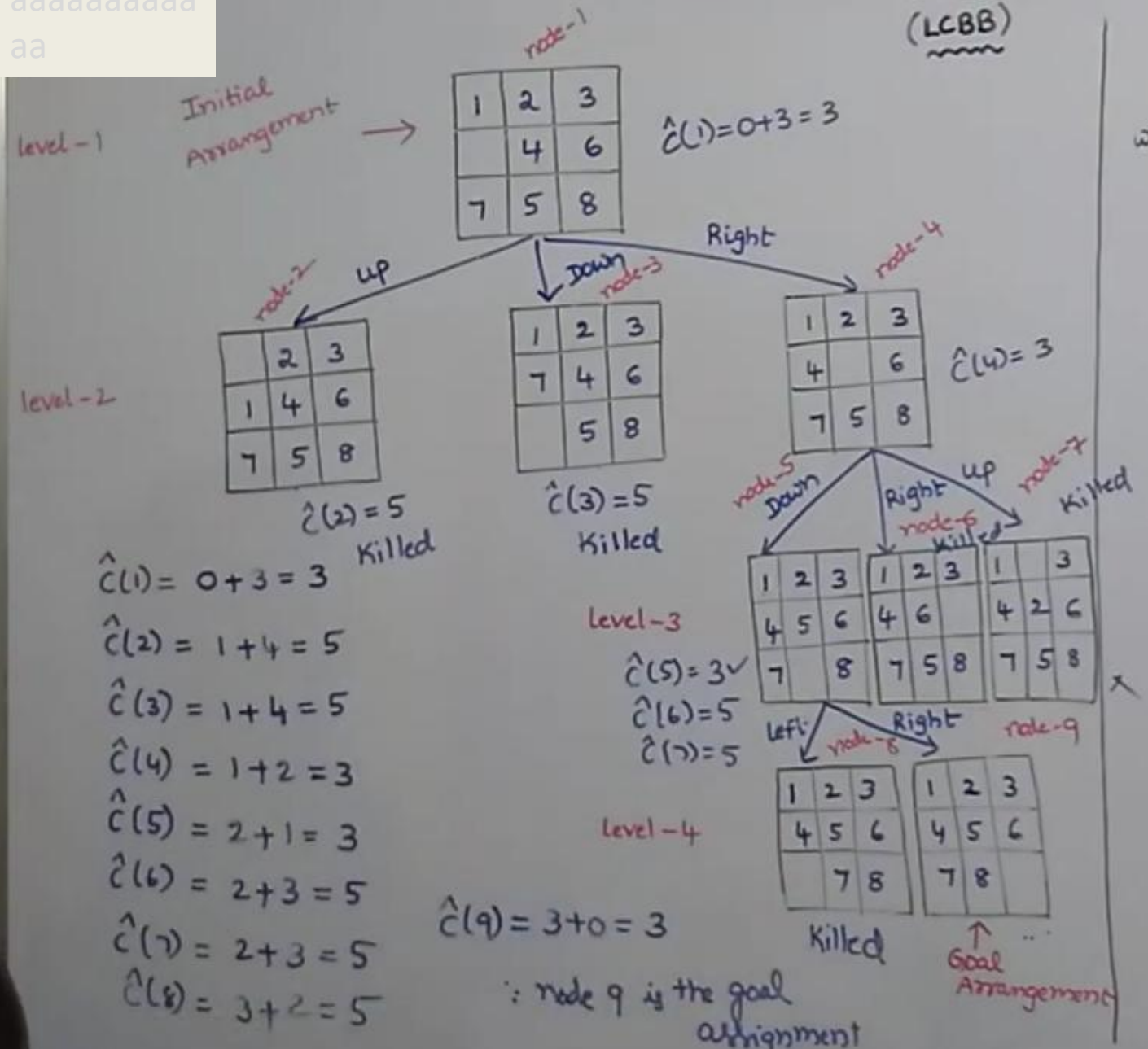
$$\hat{c}(x) = f(x) + \hat{g}(x)$$

where

$\hat{c}(x)$ = lower Bound cost of node 'x'

$f(x)$ = length of the path from root node to node 'x'.

$\hat{g}(x)$ = Number of non-blank tiles which are not in their goal position.



$\hat{c}(1) = 0+3 = 3$

$\hat{c}(2) = 1+4 = 5$

$\hat{c}(3) = 1+4 = 5$

$\hat{c}(4) = 1+2 = 3$

$\hat{c}(5) = 2+1 = 3$

$\hat{c}(6) = 2+3 = 5$

$\hat{c}(7) = 2+3 = 5$

$\hat{c}(8) = 3+2 = 5$

$\hat{c}(9) = 3+0 = 3$

∴ node 9 is the goal assignment